Section 1 — Incident Header

Incident ID: INC-20260419-005
Date: 19 April 2026
Time Opened (IST): 09:42 IST
Severity: High
Client: RetailMax Pvt Ltd
Industry: Retail
L1 Analyst: Arjun Mehta
L2 Analyst: Rohan Desai
L3 Analyst: Nisha Kapoor
Current Status: Under Investigation
Sentinel Incident Number: 548921

Section 2 — Client & Environment Context

RetailMax Pvt Ltd is one of India's rapidly expanding retail chains, operating across 40+ cities with a hybrid model that combines physical storefronts and a high-volume e-commerce platform. Their revenue stream depends heavily on uninterrupted point-of-sale systems, online transaction processing, and centralized inventory management — making uptime and data integrity critical to business operations.

From an infrastructure standpoint, RetailMax runs a hybrid environment. Their headquarters in Mumbai houses two primary domain controllers and a set of on-premise application servers that manage store synchronization and billing workflows. These systems are tightly integrated with Microsoft 365 E5 services, giving them full coverage across Defender for Endpoint, Defender for Office 365, Defender for Identity, and Microsoft Sentinel. Their endpoint estate consists of approximately 1,200 Windows 10/11 systems — including corporate laptops, finance desktops, and store-level POS terminals — along with 120 Windows Server instances supporting backend services.

Identity is managed through a hybrid Active Directory synced to Azure AD via Azure AD Connect. Conditional Access policies enforce MFA across all users, though legacy protocols are still permitted for a subset of older POS integrations — a known risk that has been documented but not yet fully remediated. Their Azure footprint includes storage accounts for transaction logs, a customer analytics platform, and several APIs exposed to partner vendors.

RetailMax is subject to PCI-DSS compliance due to credit card processing, as well as ISO 27001 controls. Any compromise involving endpoint execution or potential lateral movement is treated seriously due to the risk of cardholder data exposure and disruption of retail operations.

For this particular week, the finance and procurement teams are handling quarterly vendor reconciliation — meaning multiple employees are receiving Excel-based invoices and reports from external vendors. This temporary increase in document exchange creates a slightly elevated exposure surface for macro-based threats.

Section 3 — Attack Narrative

The attacker did not begin with code — they began with familiarity.

They had spent days profiling RetailMax's vendor ecosystem, identifying patterns in communication between procurement teams and third-party suppliers. One vendor stood out: a mid-sized logistics partner that regularly sent Excel-based shipment reconciliation reports at the end of each quarter. The format was predictable. The timing was consistent. More importantly, it was trusted.

The attacker registered a lookalike domain — close enough that, at a glance, it would pass unnoticed in a busy inbox. They crafted an email that mimicked prior correspondence: same subject structure, same tone, even copied signature formatting. Attached to it was an Excel file named *"Q4_Shipment_Reconciliation_Final.xlsm."*

Inside that file, hidden behind what appeared to be standard spreadsheet data, was a macro.

The macro was designed with restraint. It did not immediately execute anything noisy. Instead, it waited for user interaction — a click on "Enable Content." The attacker knew that in finance workflows, this prompt is often ignored or bypassed without hesitation.

At 09:31 IST, the file was opened on a corporate workstation.

The moment macros were enabled, the embedded VBA script triggered silently. It spawned a hidden instance of cmd.exe through the Excel process, carefully avoiding any visible window. From there, a second-stage command executed — a PowerShell payload passed as an encoded string.

The payload was obfuscated, layered just enough to evade casual inspection. Once decoded in memory, it attempted to establish an outbound connection to an external IP address controlled by the attacker. This was not a full payload delivery yet — it was a beacon, a handshake. A way to confirm that execution had succeeded and the host was reachable.

The attacker waited.

If the connection succeeded, the next stage would follow: a more complex payload, likely a downloader or credential harvesting script. But even without that, the initial foothold had

already been achieved — code execution inside a trusted enterprise environment, triggered by a user doing exactly what their job required.

From the attacker's perspective, the hardest part was already over. The rest would depend on how long they could remain unseen.

Section 4 — Detection & Alert

At **09:42 IST**, the alert surfaced in Microsoft Sentinel — not as an isolated anomaly, but as a pattern that Defender for Endpoint had already flagged with high confidence.

The analytics rule responsible was:

Rule Name: Suspicious Office Application Spawning Command Shell with Encoded PowerShell
Data Source: DeviceProcessEvents
Severity: High

KQL Rule Logic:

```
DeviceProcessEvents

| where TimeGenerated > ago(5m)  // Focus on recent activity window

| where InitiatingProcessFileName =~ "WINWORD.EXE"  // Office application spawning child process

| where FileName =~ "cmd.exe" or FileName =~ "powershell.exe"  // Suspicious shell execution

| where ProcessCommandLine contains "-EncodedCommand"  // Obfuscated PowerShell indicator

| project TimeGenerated, DeviceName, InitiatingProcessFileName, FileName, ProcessCommandLine, AccountName
```

The rule triggered on a specific event chain observed on endpoint **RMX-FIN-WS-023**.

Triggering Data Points:

- **Initiating Process:** WINWORD.EXE
- **Child Process:** cmd.exe → powershell.exe
- **Command Line:** Included -EncodedCommand SQBFAFgA... (Base64 encoded payload)
- **User Account:** rakesh.sharma@retailmax.com

- **Timestamp:** 09:41:27 IST

Within milliseconds of this process chain executing, Defender for Endpoint correlated an outbound network attempt originating from the PowerShell process toward IP **185.212.44.91** over HTTPS (port 443).

How the Alert Appeared in Sentinel Queue

On the primary monitor across the SOC floor — the central incident display — a new High-severity incident appeared at the top of the queue:

- **Title:** *Office Application Spawned Suspicious Command Shell with Encoded Payload*
- **Severity:** High
- **Entities:**
  - Account: rakesh.sharma@retailmax.com
  - Host: RMX-FIN-WS-023
  - Process: WINWORD.EXE → cmd.exe → powershell.exe
  - IP: 185.212.44.91
- **Time:** 09:42 IST

No audible alert tone — not Critical — but the yellow highlight in the queue made it stand out immediately among lower-severity noise.

This was not a single suspicious command. It was a chain — Office → shell → encoded PowerShell → outbound connection.

And that chain was exactly what the rule was built to catch.

Section 5 — L1 Triage (Arjun Mehta)

At **09:42 IST**, **Arjun Mehta** was midway through annotating a low-severity alert — a benign failed login from a known internal scanner — when his eyes flicked back to his left monitor.

A new High-severity incident had appeared at the top of the Sentinel queue.

He muttered under his breath, almost instinctively, reading the title aloud:
"Office app… spawning command shell… encoded payload…"

His mouse moved before he finished the sentence.

He clicked the incident.

The right-side panel expanded. He scanned the alert description first — Defender for Endpoint correlation, Office process lineage, encoded PowerShell execution. His eyes dropped to the entity list.

WINWORD.EXE → cmd.exe → powershell.exe.

He leaned back slightly. That sequence was wrong. Word doesn't need a command shell. And it definitely doesn't need PowerShell — not with encoded commands.

He glanced at the timestamp: **09:41 IST**. Fresh.

On his second monitor, he opened the **Entity Behavior** tab for the host **RMX-FIN-WS-023**. The timeline populated. He scrolled back 30 minutes.

09:31 — WINWORD.EXE launch.
09:31 — File opened: *Q4_Shipment_Reconciliation_Final.xlsm*.

He paused.

Excel macros. Or Word document? No — Excel file but Word process? He frowned for a second, then realized — misreading. It was Excel that initiated, but the alert shows Word? He double-checked the process name again.

WINWORD.EXE.

He opened the process details pane again. No mistake.

That was his first moment of hesitation.

"Wait… why Word, not Excel?"

He quickly switched to the raw process command line view. The file path showed a temp directory — C:\Users\rakesh.sharma\AppData\Local\Temp\...

He scribbled a note on his pad:
Temp execution — suspicious.

On his browser tab, he opened **VirusTotal** and pasted the external IP: **185.212.44.91**.

The result loaded — multiple detections. Known malicious infrastructure. Associated with C2 frameworks.

His jaw tightened slightly.

He flipped to **Defender for Endpoint portal**. Navigated to the device timeline. The process tree visualization rendered.

There it was — clear as a diagram:

```
WINWORD.EXE
└── cmd.exe
    └── powershell.exe (-EncodedCommand)
```

And below that, a network event attempt.

He hovered over it.
ConnectionStatus: Failed

That gave him pause.

“Did it actually connect… or just try?”

If the connection failed, maybe the payload didn’t execute fully. Maybe this was contained. But the process chain had already executed — the code ran.

He didn’t like ambiguity.

He checked **Entity Behavior** again — no repeated attempts yet. No lateral movement. No new processes.

But he knew this pattern. Macro → PowerShell → outbound beacon. This was textbook initial access execution.

He exhaled slowly and began typing into the **Sentinel Incident Comments**:

“Observed WINWORD.EXE spawning cmd.exe followed by PowerShell with encoded command on host RMX-FIN-WS-023. Associated outbound connection attempt to IP 185.212.44.91 (flagged malicious via VirusTotal). Initial review shows execution occurred post document open from temp directory. Connection attempt status unclear — requires deeper validation.”

He paused for a second before choosing classification.

Not False Positive.
Not Benign.

This was real — even if incomplete.

He switched to Microsoft Teams, SOC-Incidents channel, and typed:

Escalation Message (09:48 IST)

@Rohan Desai
High severity incident — suspected macro-based malware execution.

What fired: Defender alert — WINWORD.EXE spawning cmd.exe → PowerShell with -EncodedCommand

Key observations:

- Host: RMX-FIN-WS-023 (Finance user)
- File opened from temp directory around 09:31 IST
- PowerShell attempted outbound connection to 185.212.44.91 (VT flagged malicious)
- Process chain confirmed via Defender timeline

Initial assessment: Likely macro-triggered payload execution. Unsure if C2 connection succeeded — needs validation.

Request:

- Confirm outbound connection success via DeviceNetworkEvents
- Decode PowerShell payload from command line
- Check for any follow-on activity or persistence

Incident updated with notes.

He hit Enter, then leaned back, eyes still on the process tree.

“Macro,” he muttered quietly.
“Always starts with a macro.”

Section 6 — L2 Investigation (Rohan Desai)

At **09:49 IST**, the notification ping appeared in Microsoft Teams.

Rohan Desai didn’t open Sentinel immediately.

He read Arjun’s escalation first — slowly, line by line.

Office → cmd → PowerShell → encoded command → outbound IP.

He nodded once.

“Clean chain,” he said quietly to himself. “No noise.”

Only then did he turn to his center monitor and open the incident.

Initial Assessment & Query Planning

He expanded the investigation graph. The visual confirmed what Arjun had described — single host, single user, single execution chain, one external IP.

No lateral spread — yet.

He opened Notepad++ on his second monitor and began typing:

Investigation Questions:

1. Did the PowerShell process successfully establish outbound connection?
2. What exactly was the encoded payload?
3. Was there any follow-on execution or persistence attempt?
4. Was this isolated or part of a broader campaign?

He started with the network question.

Query 1 — Did the outbound connection succeed?

```
DeviceNetworkEvents
| where TimeGenerated between (datetime(2026-04-19 09:30:00) .. datetime(2026-04-19 10:00:00))  // Investigation window
| where DeviceName == "RMX-FIN-WS-023"  // Affected host
| where RemoteIP == "185.212.44.91"  // Suspected C2 IP
| project TimeGenerated, InitiatingProcessFileName, InitiatingProcessCommandLine, RemoteIP, RemotePort, ActionType
```

Result:

- One event at **09:41:29 IST**
- **ActionType:** ConnectionFailed

Rohan leaned closer.

“Failed… but why?”

He scrolled further.

No retries. No alternate IPs.

“Either blocked by network control… or payload incomplete.”

He wrote in his notes:
No successful C2 — initial beacon attempt failed.

Query 2 — What was the PowerShell payload?

```
DeviceProcessEvents

| where DeviceName == "RMX-FIN-WS-023"

| where TimeGenerated between (datetime(2026-04-19 09:30:00) .. datetime(2026-04-19 10:00:00))

| where FileName == "powershell.exe"

| project TimeGenerated, ProcessCommandLine, InitiatingProcessFileName
```

The result showed the full encoded command string.

He copied it.

On his third monitor, he opened **CyberChef** and pasted the Base64 string. Applied “From Base64.”

The decoded output appeared.

A PowerShell one-liner:

- Create WebClient
- Attempt download from https://185.212.44.91/update.ps1
- Execute in memory

Rohan exhaled slowly.

“Stage two downloader,” he said.

But it never executed — because connection failed.

Pivot — Any secondary execution?

He ran another quick scan in DeviceProcessEvents.

No new processes spawned after PowerShell.

No scheduled tasks. No registry persistence.

Clean cut.

Attack Timeline Reconstruction

Timestamp (IST)	Event	Data Source	MITRE Technique
09:31:02	Malicious document opened	DeviceProcessEvents	T1204 (User Execution)
09:41:27	WINWORD.EXE spawns cmd.exe	DeviceProcessEvents	T1059.003
09:41:28	cmd.exe spawns PowerShell (encoded)	DeviceProcessEvents	T1059.001
09:41:29	PowerShell attempts outbound connection	DeviceNetworkEvents	T1071.001
09:41:29	Connection fails	DeviceNetworkEvents	-

IOC Table

IOC	Type	Source	Enrichment Result
185.212.44.91	IP	DeviceNetworkEvents	Malicious — flagged by VirusTotal, associated with C2 infrastructure
update.ps1	File path (remote)	Decoded payload	Not retrieved — likely stage 2 payload
EncodedCommand string	Command	DeviceProcessEvents	Obfuscated PowerShell downloader

---

MITRE ATT&CK Mapping

- **T1204** — User Execution (macro-enabled document)
- **T1059.003** — Windows Command Shell
- **T1059.001** — PowerShell execution
- **T1071.001** — Web protocol C2 attempt

Conclusion

Rohan leaned back in his chair.

“This didn’t fail by design,” he said quietly. “It failed by timing.”

The macro executed successfully. The PowerShell payload ran. The only thing that prevented escalation was the failed outbound connection.

Had that succeeded, the system would have downloaded and executed a second-stage payload.

This was a **partial compromise — contained by network failure, not detection**.

Containment Recommendations

1. **Isolate Host (RMX-FIN-WS-023)**
    - Use Defender for Endpoint “Isolate Device” action immediately.
2. **Block IP (185.212.44.91)**
    - Add to Azure Firewall deny list
    - Add to Defender indicator block list
3. **User Awareness + Reset**
    - Force password reset for rakesh.sharma
    - Confirm no credential harvesting occurred
4. **Hunt Across Environment**
    - Search for same encoded command across all endpoints

    - Query for similar WINWORD → PowerShell chains

5. **Email Trace**
    - Identify and purge similar phishing emails from all mailboxes via Defender for O365

He documented everything in Sentinel.

Then he typed a final note:

“Execution confirmed. C2 stage blocked due to failed outbound connection. No persistence observed. Recommend immediate containment and environment-wide hunting for similar payload.”

He paused for a moment, then added one more line:

“This would have progressed if not interrupted.”

And that, to him, was the most important finding.

Section 7 — L3 Detection Tuning (Nisha Kapoor)

By **11:05 IST**, the SOC floor had settled back into its steady rhythm. The yellow sticky note for the RetailMax incident had already been pulled off the board — containment was in progress.

At the back row, seated behind Rohan’s workstation, **Nisha Kapoor** opened the incident report on her ultra-wide monitor.

She didn’t skim.

Her eyes moved slowly across Rohan’s notes — process chain, encoded payload, failed outbound connection. She paused at one line:

“This would have progressed if not interrupted.”

She leaned back slightly.

“That’s the problem,” she said quietly.

Not that the attack happened — but **when it was detected**.

She reached for her keyboard and opened the **Microsoft Sentinel Analytics Rules editor**. In the search bar, she typed the rule name:

“Suspicious Office Application Spawning Command Shell with Encoded PowerShell”

The rule opened. She read the KQL as-is.

Before-State KQL Rule

```
DeviceProcessEvents

| where TimeGenerated > ago(5m)

| where InitiatingProcessFileName =~ "WINWORD.EXE"

| where FileName =~ "cmd.exe" or FileName =~ "powershell.exe"

| where ProcessCommandLine contains "-EncodedCommand"

| project TimeGenerated, DeviceName, InitiatingProcessFileName, FileName,
ProcessCommandLine, AccountName
```

She tapped her pen lightly against the desk.

“Too late,” she said.

The rule was waiting for **encoded PowerShell execution** — the final stage of the macro chain. By the time this fired, the payload had already run.

She opened Notepad++ and typed a single line at the top:

Weakness: Detection triggers only at encoded PowerShell stage, missing earlier macro-to-shell execution pattern.

She looked back at the investigation timeline.

The earliest reliable signal wasn’t PowerShell — it was **Office spawning a shell process at all**.

That was abnormal enough.

She began rewriting.

After-State Improved KQL Rule

```
DeviceProcessEvents

| where TimeGenerated > ago(5m)
```

```
| where InitiatingProcessFileName in~ ("WINWORD.EXE", "EXCEL.EXE")  // Include both Word and Excel as macro sources

| where FileName in~ ("cmd.exe", "powershell.exe")  // Shell processes spawned by Office apps

| extend SuspiciousFlag = case(

   ProcessCommandLine contains "-EncodedCommand", "EncodedPowerShell",  // High confidence malicious

   ProcessCommandLine contains "powershell", "PlainPowerShell",  // Medium confidence

   "ShellSpawn"  // Baseline suspicious behavior

)

| summarize EventCount = count(), Flags = make_set(SuspiciousFlag)

   by DeviceName, AccountName, InitiatingProcessFileName, bin(TimeGenerated, 5m)

| where EventCount >= 1  // Trigger even on single suspicious chain

| project TimeGenerated, DeviceName, AccountName, InitiatingProcessFileName, Flags
```

She read it again, line by line.

This version didn't wait for the payload to fully reveal itself. It treated **Office spawning a shell** as the primary anomaly — and layered severity based on how suspicious the command line was.

She nodded once.

"This fires earlier," she said. "Even before the payload calls home."

She added comments directly into the rule draft:

- Include Excel.EXE (macro origin missed previously)
- Detect shell spawn regardless of encoding
- Add classification logic for triage prioritization

Then she opened the incident timeline again and mentally replayed it.

With this rule, the alert would have fired at **09:41:27 IST — the moment cmd.exe spawned**.

A few seconds earlier than before.

Not a huge gap in time — but in attacks like this, seconds matter.

Explanation of Improvement

The updated rule shifts detection from **payload-specific behavior (encoded PowerShell)** to **behavioral anomaly (Office spawning shell processes)**. This allows the SOC to detect malicious macro execution earlier in the attack chain, even if the attacker changes obfuscation techniques or avoids encoded commands altogether.

It also introduces contextual classification (EncodedPowerShell, PlainPowerShell, ShellSpawn) to help L1 analysts quickly assess severity without losing visibility.

In plain terms:
The new rule catches the attack when it starts behaving suspiciously — not when it's already executing its payload.

And that difference is what prevents near-misses from becoming incidents.

Section 8 — Final Incident Report

Part A — Executive Summary

A malicious document sent to an employee triggered unauthorized code execution on a company workstation. The file appeared legitimate but contained hidden instructions that attempted to connect to an external system controlled by an attacker.

The connection attempt failed, which prevented further compromise. Security monitoring tools detected the activity quickly, and the affected system was isolated before any damage occurred.

No data loss or system impact has been identified. Preventive measures have been implemented to reduce the likelihood of similar incidents in the future.

Part B — Full Technical Summary

Incident Title: Macro-Based Malware Execution via Malicious Document
Client: RetailMax Pvt Ltd
Severity: High
Status: Contained

Attack Type

- Initial Access via Malicious Document (Macro-enabled file)
- Execution via Command Shell and PowerShell

MITRE ATT&CK Techniques

- T1204 — User Execution
- T1059.003 — Windows Command Shell
- T1059.001 — PowerShell
- T1071.001 — Web Protocols (C2 attempt)

Timeline Summary

Time (IST)	Event
09:31	User opened macro-enabled document
09:41:27	WINWORD.EXE spawned cmd.exe
09:41:28	cmd.exe spawned PowerShell with encoded payload
09:41:29	PowerShell attempted outbound connection
09:41:29	Connection failed
09:42	Sentinel alert generated
09:48	Escalation to L2
10:05	Containment actions initiated

Impact Assessment

- No successful command-and-control communication
- No secondary payload downloaded
- No persistence mechanisms established
- No lateral movement observed
- Incident limited to single endpoint

IOC List

IOC	Type	Description
185.212.44.91	IP Address	Malicious C2 infrastructure
Encoded PowerShell Command	Command	Obfuscated downloader script
update.ps1	Remote File	Intended second-stage payload

Containment Actions Taken

- Endpoint **RMX-FIN-WS-023** isolated via Defender for Endpoint
- Malicious IP **185.212.44.91** blocked in Azure Firewall and Defender IOC list
- User account password reset enforced
- Threat hunting initiated across all endpoints for similar execution patterns
- Email trace and purge initiated for similar phishing attempts

---

Lessons Learned

- Macro-based attacks remain effective in high-trust business workflows
- Detection relying solely on payload indicators delays response
- Early behavioral indicators (Office spawning shell) are critical
- Network failure prevented escalation — not a reliable defense

---

Action Items

Action	Owner	Priority	Target Date
Deploy updated Sentinel detection rule (Office → shell correlation)	Nisha Kapoor	High	21-Apr-2026

Disable or restrict macro execution for external documents via GPO	IT Security Team	High	25-Apr-2026
Conduct phishing awareness training for finance and procurement teams	Security Awareness Team	Medium	30-Apr-2026
Implement attachment sandbox detonation policy for all inbound Excel files	Email Security Team	High	23-Apr-2026
Review and restrict legacy protocol exceptions in Conditional Access	Identity Team	Medium	28-Apr-2026

The incident has been fully investigated and contained. No further malicious activity has been observed at the time of report closure.